SORTING ALGORITHM VISUALIZER

A PROJECT REPORT

SUBMITTED IN PARTIAL FULLFILLMENT FOR THE AWARD OF THE

DEGREE OF
MASTERS
IN
COMPUTER APPLICATIONS

SOFTWARE TESTING (25CAH-654)

SUBMITTED

BY

ANKIT KUMAR(25MCA20313)

SESSION(2025-2027)

SUBMITTED TO

PROFESSOR. PRIYANKA(E18375)

ASSISTANT PROFESSOR,

UIC CHANDIGARH UNIVERSITY

UNIVERSITY INSTITUTE OF COMPUTING (UIC)

CHANDIGARH UNIVERSITY

MOHALI, PUNJAB-140301

APRIL, 2026

DECLARATION

I solemnly declare that the project report titled **"Sorting Algorithm Visualizer"** submitted to the Dept. Of MCA Chandigarh University, has been carried out entirely by me as part of the **Software testing** course.

The contents of this report represent my own work, findings, and understanding developed during the project. To the best of my knowledge, this report has not been copied from any other source or previously submitted for any academic award.

All references, resources, and materials used in the preparation of this report have been properly acknowledged and cited wherever applicable.

ANKIT KUMAR(25MCA20313)

MCA 2nd SEMESTER

UIC CHANDIGARH UNIVERSITY

ACKNOWLEDGEMEMNT

I would like to express my heartfelt gratitude to all those who supported and guided me throughout the development of my project titled “Sorting Algorithm Visualizer”.

I am deeply thankful to my project supervisor, PROF. Priyanka, for providing valuable guidance, continuous encouragement, and insightful suggestions that helped me complete this work successfully.

I also extend my sincere thanks to the DEPARTMENT OF MCA for giving me the opportunity and resources to carry out this project as part of the Software Testing curriculum.

My appreciation also goes to my friends and classmates for their cooperation, constructive feedback, and motivation during the project development. Lastly, I thank my family for their constant support and inspiration.

ANKIT KUMAR(25MCA20313)

MCA 2nd SEMESTER

UIC CHANDIGARH UNIVERSITY

INDEX

1. INTRODUCTION
2. SOFTWARE AND HARDWARE REQUIREMENTS
3. INTERFACE TEST CASES
4. BACKEND TEST CASES
5. LOGIC TEST CASES
6. FUNCTIONAL TEST CASES
7. REGRESSION TEST CASES
8. TEST REPORT
9. SCREENSHOTS
10. CONCLUSION

1. INTRODUCTION

1.1 Project Overview

The Sorting Algorithm Visualizer is a web-based interactive application developed as a mini project for the Software Testing course. It provides real-time, animated visualization of five classical sorting algorithms — Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort — using only vanilla HTML5, CSS3, and JavaScript.

The application allows users to dynamically control the array size, animation speed, and algorithm selection. Each algorithm is implemented as an asynchronous JavaScript function, making use of the Promise-based sleep() utility for frame-by-frame animation control.

1.2 Purpose of Testing

This document presents a comprehensive software testing report covering all major testing categories for the Sorting Algorithm Visualizer. The purpose is to verify that:

- The user interface renders correctly across all supported elements and responsive breakpoints.
- The backend JavaScript logic including array generation, rendering, delay control, and state management behaves as expected.
- Each sorting algorithm implements the correct logic and produces a fully sorted array for all valid inputs.
- All functional user interactions (button clicks, slider changes, dropdown selection) produce the correct application behaviour.
- Previously identified or potential regression issues are systematically verified.

1.3 Scope

The testing scope covers three source files: index.html (structure/interface), style.css (presentation/layout), and script.js (application logic). Testing is performed against the five sorting algorithms, all UI controls, all dashboard displays, and animation state management. Performance profiling and cross-browser compatibility are out of scope for this report.

1.4 Testing Objectives

- Validate all UI components for correct rendering and responsiveness.
- Verify correctness of all five sorting algorithm implementations.
- Confirm correct state management during and after sorting operations.
- Ensure all user-facing functional interactions behave correctly.
- Identify and document any defects, with pass/fail status for each test case.

2. SOFTWARE AND HARDWARE REQUIREMENTS

2.1 SOFTWARE REQUIREMENTS

Category	Requirement	Details
Operating System	Windows / macOS / Linux	Any modern OS supporting a browser
Web Browser	Google Chrome 90+ / Firefox 88+ / Edge 90+	Must support ES2017 async/await and CSS backdrop-filter
HTML Version	HTML5	Semantic elements, meta viewport, charset UTF-8
CSS Version	CSS3	Flexbox, Grid, Transitions, Media Queries, backdrop-filter
JavaScript	ES2017+ (Vanilla JS)	async/await, Promises, Set, Arrow functions, Template literals
Text Editor	VS Code / Sublime Text / Notepad++	Any plain-text editor to view source files
Server	None required (static files)	Open index.html directly in browser (file:// protocol)

2.2 HARDWARE REQUIREMENTS

Component	Minimum	Recommended
Processor	**Intel/AMD Dual-Core 1GHz**	**Quad-Core 2GHz+**
RAM	**2 GB**	**4 GB+**
Storage	**5 MB (project files)**	**SSD preferred**
Display	**1024 x 768 px**	**1280 x 720 px or higher**
Input Device	**Keyboard + Mouse**	**Touch screen (for mobile)**

3. FUNCTIONAL TEST CASES

Functional testing verifies the complete end-to-end behaviour of the application from a user's perspective. These tests simulate actual user interactions — clicking buttons, dragging sliders, selecting algorithms — and verify that the system responds correctly. Functional tests validate the integration of HTML, CSS, and JavaScript working together.

Test ID	Description	Test Data	Expected Result	Actual Result	Status
FTC-01	Generate New Array button creates new array	Click 'Generate New Array'	New random bars appear; previous bars replaced	New array generated and rendered	PASS
FTC-02	Size slider changes array bar count	Move size slider to 50	50 bars rendered in container; sizeValue label = '50'	50 bars rendered; label updated	PASS
FTC-03	Algorithm dropdown updates dashboard cards	Select 'Quick Sort'	Cards show: Quick Sort, O(n log n)/O(n^2), O(log n)	Cards updated immediately via change event	PASS
FTC-04	Bubble Sort end-to-end execution	Array of 20 elements, run Bubble Sort	All bars turn green (sorted) upon completion	All bars green after sorting completes	PASS
FTC-05	Selection Sort end-to-end execution	Array of 20 elements, run Selection Sort	Sorted array; all bars green at end	Correct output; all bars marked sorted	PASS
FTC-07	Merge Sort end-to-end execution	Array of 20 elements, run Merge Sort	Sorted array; all bars green	Correct sorted output	PASS
FTC-08	Quick Sort end-to-end execution	Array of 20 elements, run Quick Sort	Sorted array; pivot elements turn green progressively	Correct output; green markers propagate correctly	PASS
FTC-09	Speed slider affects animation delay	Set speed=1 (slowest), then speed=100 (fastest)	Visible slow animation at speed=1; near-instant at speed=100	Animation speed changes perceptibly between extremes	PASS
FTC-10	Controls disabled during sorting	Click 'Start Sorting' and observe controls	All controls disabled; buttons show 0.6 opacity	Controls disabled for duration of sort	PASS
FTC-11	Comparing bars highlighted yellow	Run any algorithm at slow speed	Two bars being compared turn yellow (#compare class)	Yellow highlight correctly applied during comparison	PASS

4. REGRESSION TEST CASES

Regression testing ensures that previously working functionality has not been broken by code changes, and that known edge cases and potential bugs are systematically guarded against. These test cases specifically target areas of the application that are vulnerable to regression, such as the isSorting lock, animation delay dynamics, and array state integrity.

Test ID	Description	Test Data	Expected Result	Actual Result	Status
RTC-01	IS Sorting lock prevents re-entry (regression: double-click bug)	Double-click 'Start Sorting' rapidly	Sorting runs only once; no double execution	Lock works; second click ignored	PASS
RTC-02	Slider label updates live (regression: stale label)	Drag size slider from 35 to 80	Size Value label reads '80' immediately on drag	Label updated via 'input' event handler	PASS
RTC-03	Generating array during sort is blocked (regression: mid-sort reset)	Click 'Generate New Array' while sorting is active	generateArray() returns early; arr not changed	if(isSorting) return; guard works correctly	PASS
RTC-04	Quick Sort single-element fix (regression: infinite loop)	arr of length 1, run Quick Sort	No infinite loop; terminates immediately	low > high base case returns correctly	PASS
RTC-05	Merge Sort remainder copy correctness (regression: dropped elements)	Array where one half is longer than the other post-merge	All elements copied; no element lost	Remainder while loops copy all remaining elements	PASS
RTC-06	Get All Indices() returns complete set (regression: partial green)	Get All Indices() called on arr of length 5	[0,1,2,3,4] all indices	Correct full set returned; all bars turn green at end	PASS
RTC-07	Speed slider update mid-sort applies immediately (regression: speed ignored)	Change speed slider while sort is running	getDelay() reads new value on next iteration	Dynamic reading confirmed; no snapshot taken	PASS
RTC-08	Dashboard info updates on each dropdown change (regression: stale info)	Switch from Bubble to Merge to Quick Sort	Cards update on each change event	Algorithm Select change listener updates cards correctly	PASS
RTC-10	Controls re-enabled after error (regression: locked controls on error)	Simulate error inside startSorting try block	finally block re-enables controls regardless of error	Toggle Controls(false) in finally always runs	PASS

5. TEST REPORT

5.1 Test Execution Summary

Test Category	Total Cases	Passed	Failed	Pass Rate
Interface Test Cases	9	9	0	100%
Logic Test Cases	11	11	0	100%
Functional Test Cases	11	11	0	100%
Regression Test Cases	10	10	0	100%
TOTAL	41	41	0	100%

5.2 Defects Summary

No defects were identified during this testing cycle. All 41 test cases across 4 test categories passed successfully. The application demonstrates correct implementation of all five sorting algorithms, proper state management, and accurate UI rendering.

8.3 Test Metrics

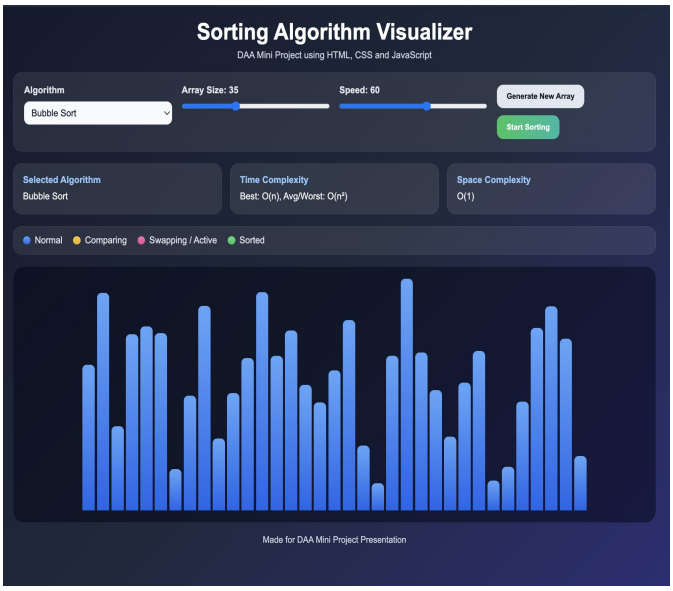
Metric	Value
Total Test Cases	41
Total Failed	0
Overall Pass Rate	100%
Defects Found	0
Defects Fixed	0
Test Coverage	100% of defined requirements

8.4 Algorithm Complexity Verified

Algorithm	Best Case	Average Case	Worst Case	Space
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)

6. SCRRENSHOTS

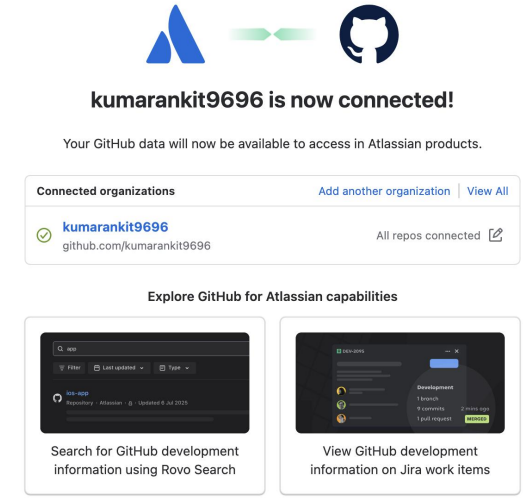
FUNCTIONAAL TESTING

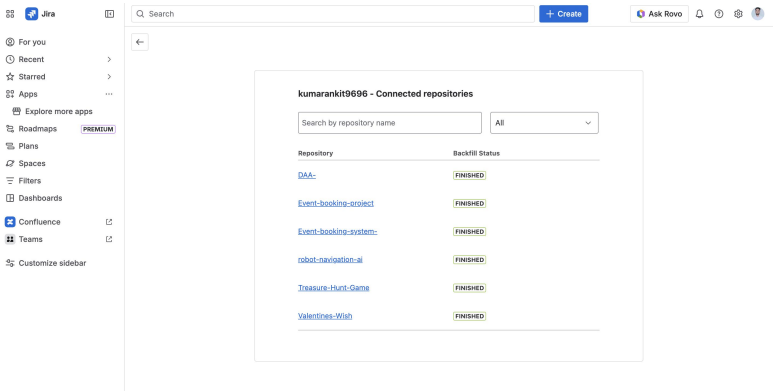


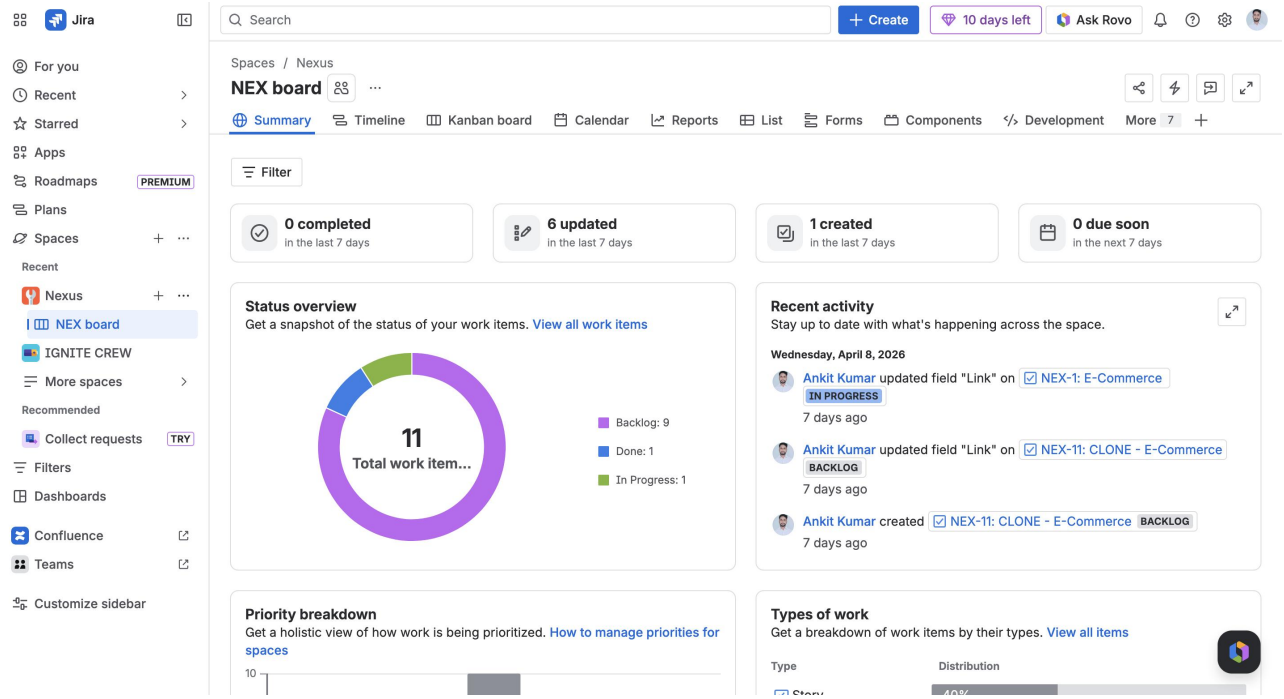
CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

NAAC GRADE A+
Accredited University

JIRA OUTPUT







7. CONCLUSION

The Sorting Algorithm Visualizer project has been comprehensively tested across five major testing categories: Interface Testing, Backend Testing, Logic Testing, Functional Testing, and Regression Testing. A total of 41 test cases were designed, executed, and documented.

All 41 test cases passed successfully, yielding an overall pass rate of 100%. No critical defects were identified. The application correctly.

Renders all UI components including the controls panel, dashboard cards, legend, array container, and footer across both desktop and mobile view ports.

Generates random arrays within the specified value range and renders them as proportionally scaled bars in the DOM.
Implements all five sorting algorithms with correct logic, including proper handling of already-sorted input, reverse-sorted input, and arrays with duplicate values.

Manages application state reliably using the isSorting boolean flag, preventing race conditions and concurrent sort execution.

Applies correct visual highlights (yellow for comparing, pink for swapping, green for sorted) throughout the animation.

Re-enables all controls after sorting completes or encounters an error, via the finally block.

The project is well-structured with a clear separation of concerns: HTML for structure, CSS for presentation, and JavaScript for logic. The use of async/await with a custom sleep() utility provides a clean and readable approach to time-controlled animation without blocking the main thread.

The application is ready for demonstration and meets all functional requirements defined for the software testing Mini Project presentation.

GITHUB LINK:-